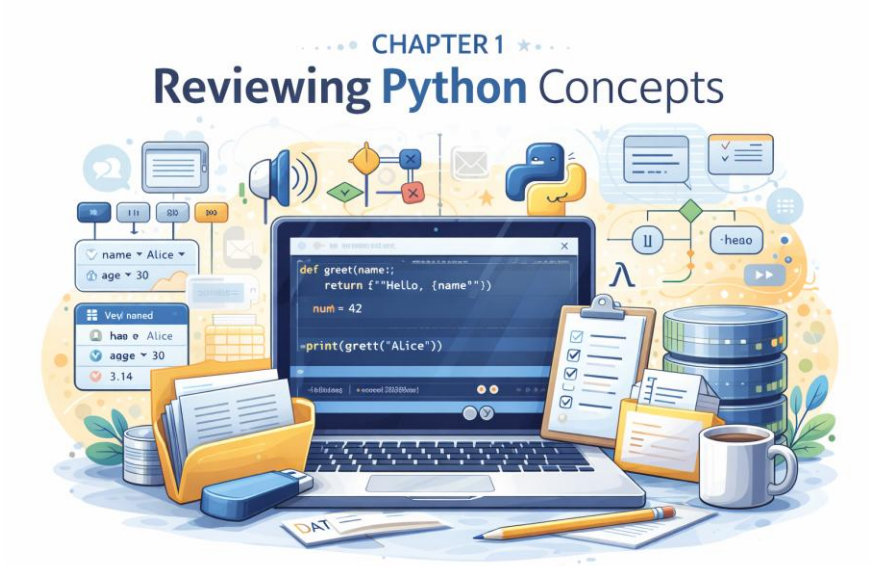

The following chapter was generated using ChatGPT and further edited by Lucas Hartman

Chapter 1 – Python Basics Review

1. Printing in Python

The print() function is used to display information. You can control how items are separated with the sep parameter and how lines end with the end parameter. This is especially useful when formatting output like tables, lists, or sentences.

Key points:
- `sep` controls what goes between multiple items (default is space – “ “).
- `end` controls how each print statement ends (default is newline – “\n”).

```
print("Hello", "World", sep=", ")  # Outputs: Hello,World
print("Hello", end=" "); print("World") # Outputs: Hello World
```

When printing and building string literals we also have the use of escape characters. These can be put into string literals to add in special formatting. Note: we are saying “into a string” because you are building a string to then print it out. Or you could be assigning the string to a variable.

- \n inserts a new line into the string
- \t inserts a tab into the string
- \’ inserts a single quote mark into the string
- \” inserts a double quote mark into the string
- \\ inserts a backslash into the string

2. Variable, Input and Casting

All these topics are how we get, store and manipulate data.

2.1 Variables

Variables are a way to store information within our programs. To create a variable, you only need to start typing out the variable name then use the assignment operate '=' to give it a value.

```
age = 16

first_name = "John"

last_name = "Smith"
```

Naming the variables should be meaningful to what information they are holding. As above you can deduce, the variable 'age' is a value of someone's age, and same thing with first and last name variables.

Naming conventions – per the PEP 8 guide, we should use underscores making variable names and function names. Other languages like Java, C++, etc. like the camel case method.
- camel case: `firstName, lastName`
- underscore: `first_name, last_name`

Both options above make it easier to read the variable names. In Python they want you to follow their style guides.

Constant variables are variables that don't change. These are values you can use in calculations or other operations throughout your code. Constants are coded in all caps and values shouldn't be changed. Python doesn't have any way to set a constant to not be able to change (Like other languages that make values "final")

```
DAYS_IN_JANUARY = 31

MINUTES_IN_HOUR = 60

DEFAULT_USER = "Guest"
```

2.2 Input

Typically, in a program we don't just code literal values and instead want to get input from the user. For console-based programs we use the input() function. This accepts a string literal for a prompt to print out

```
first_name = input("Enter your first name: ")

last_name = input("Enter your last name: ")
```

2.3 Casting

Since `input()` always returns a string, you often need to cast input into numbers. Casting directly in the input call is cleaner and prevents bugs. For example, if you need an integer right away, use `int(input(...))`. Always ensure you handle errors later using try/except.

```
age = int(input("Enter your age: "))

height = float(input("Enter your height in meters: "))
```

Note that casting is shown here using the input() function but often times you may be casting a variable not just from input. Possibly you are reading in from a file, all that data comes in a strings, if you need the data as a different type, cast it.

```
*** Code that opened a file, saved to 'inputFile' variable ***

line = inputFile.readline() # reads the next line

age = int(line) # converts to an integer
```

Common casting functions:
- int()
- float()
- str()
- bool()

Always validate or catch errors when casting – more on that later.

2.4 Printing Variables

You may need to print out variables along with text. You will want to understand the best way to do so.

Concatenations with the '+'

```
first_name = input("Enter your first name: ")

print("Hello, " + first_name + " glad to see you.")
```

Note that the data type here for 'firstName' is a string. If you ever involve using non string variables like int or float, you will need to cast those using the str() function.

Use the separator in a print()

```
first_name = input("Enter your first name: ")

print("Hello,", first_name, "glad to see you.")
```

Use f strings to print out values.

```
first_name = input("Enter your first name: ")

print(f"Hello, {first_name} glad to see you.")
```

Key Points
- You must put an 'f' in front of the string
- Variables are defined in curly brackets {}. Not set variable limit.
- Not only for print, can be used for setting a variable value, or a property of an object.

There are of course many more uses of f-string especially with formatting variables for output

Operators
- Must put a colon after variable name to start formatting values
- To round numbers use a period, number of digits, and 'f' for float - .2f round two decimals
- If you want to print commas for thousands marker put a comma ',' BEFORE any rounding
- You can setup a field width for printing by putting a number of spaces BEFORE any rounding or commas are added in

```
num = 2175.678865

print(f"The number: {num}") #prints out "The number: 2175.678865"

print(f"The number: {num:.2f}") #prints out "The number: 2175.68"

print(f"The number: {num:10,.3f}")
                                #prints out "The number:   2175.679"
                                # 10 spaces to fill the number
```

2.5 Math Operations

Lastly we should review math operations that we can do with variables.

Operators

- `+` use the plus as the addition operator
- `-` use the dash as the subtraction operator
- `*` use the asterisk as the multiplication operator
- `/` us the forward slash as the division operator (decimal numbers)
- `//` use two forward slashes together for integer division (no decimals)
- `%` use the percent as as the modulus operator (remainder of division)
- `**` use two asterisks as the exponent operator
- `import math` use the math module to call function like sqrt(), floor(), fabs() to help with math calculations

2. If/Else Conditionals

Conditionals let your program branch into different behaviors. Use `if` for the first condition, `elif` for alternatives, and `else` for the default case.

```
x = int(input("Enter the temperature: "))

if temp > 90:
    print("It's hot!")

elif temp < 32:
    print("It's freezing!")

else:
    print("It's nice out.")
```

You can use logical operations `and`, `or` and `not` to combine multiple decisions

```
if temp > 32 and temp < 90:
    print("This is some good weather.")
```

Remember to watch your indentation, python uses it for blocks of code. Especially when you get into functions, or nested decision structures.

Key Points:
-and checks if all expressions are true, then run code
-or checks that at least one expression is true, run code
-not negates the expression, if true, run code

3. Loops

Loops allow repetition in code. This makes it easier that if you have statements that repeat, you can simply implement a loop instead of copying and paste the same line over and over.

3.1 While Loops

Use `while` to continue until a condition is no longer true. Great for not knowing how many times the loop will run. See later in the chapter with a menu style looping program.

```
password = ""
while password != "secret":
    password = input("Enter password: ")
```

As in the example above, while loops are great for checking user input, and looping to ask for input again. Here is another example for check input for a menu option

```
choice = 0
while choice != 4:
    # *** Code here to print a menu of 1-4 options, 4 being "Exit" ***
    choice = int(input("Choose option: ")) # get initial option
    while choice < 1 or choice > 4: # check if out of range
        print("Invalid option.")
        choice = int(input("Choose option: "))
    password = input("Enter password: ")
```

Always remember to modify the variable you are using in the condition for the while loop. Here we are having 'password' modified by getting input

Yes, you can still use a while loop to count.

3.2 For Loops

Use `for` to iterate a specific number of times or over collections.

```
colors = [“red”, “green”, “blue”] # review of lists coming up
for color in colors:
     print(color)

for i in range(3):
    print("Iteration", i)
```

The range() function returns a sequence of numbers you can use:

- range(5) → 0 to 4
- range(1, 6) → 1 to 5
- range(1, 10, 2) → 1, 3, 5, 7, 9 (step of 2)

4. Functions

Functions let you organize and reuse code.

Some functions just perform an action.

```
def greet():
     print ("Good Afternoon!”)

greet() # calling the function above
```

Other functions will return a value.

```
def greet():
     return "Good Afternoon!”

value = greet() # calling the function and saving value
print(value)
```

Sometimes you need to send values to these functions. These are called parameters. You define these within the parenthesis of the function. When you have multiple parameters, you separate them via a comma. Values are passed when calling the function in the same order they are sent.

```
def add(a, b):
     return a + b

num1 = 4
num2 = 6
total = add(num1, num2)
```

Using a `main()` function provides structure and makes programs easier to maintain. While Python doesn't require, this is industry norm to have main() functions.

```
def greet(name):
     return f"Hello, {name}!"

def main():
     print(greet("Alice"))

if __name__ == "__main__":
     main()
```

Key points:
- Use `def` to define functions.
- Include parameters in parenthesis after function name if needed
- Functions can return values using `return`.
- `if __name__ == "__main__":` ensures the program runs only when executed directly, not when imported. *More on this when discussing modules.*

5. Strings and their Methods

Strings are text and Python offers many built-in methods for cleaning and formatting them.

Examples:
- `.lower()` and `.upper()` convert case.
- `.strip()` removes whitespace.
- `.replace()` substitutes text.
- `.split()` turns a string into a list via the delimiter you note in the parenthesis.
- The `in` keyword checks if a substring exists.

```
text = "  Hello World!  "
print(text.lower())
print(text.upper())
print(text.strip())
print(text.replace("World", "Python"))
print("one,two,three".split(","))
print("Py" in "Python")
```

The above examples all print out results, you can easily assign these to variables, etc.

For more reference: [w3schools Python String Methods]

6. Lists and Dictionaries

Lists store ordered collections that have elements store at index locations. The first index is 0. Lists can contain any data type and don't have to all be the same. Use square brackets to directly access list elements.

As discussed above, we use a for loop to loop through the elements of a list.

Lists are best when order matters.

Common list methods: `.append()`, `.remove()`.

```
numbers = [1, 2, 3]
numbers.append(4) # list is not 1, 2, 3, 4
numbers.remove(2) # removes value NOT the index
print(numbers) # note that prints list including the [ ]
```

Dictionaries store key-value pairs. Dictionaries are great for looking up values by a key. Say a student ID instead of having to loop through a list and go deeper into the data.

Dictionaries let you update or add new keys by using the square brackets.

```
student = {"name": "Alice", "age": 20}
student["age"] = 21
print(student)
```

We will review and discuss more in depth features of lists and dictionaries along with other data structures in a later chapter.

7. Line Continuation in Python

In Python, each logical statement is usually written on a single line. However, some statements can become long, especially when printing multiple values, performing calculations, or passing many arguments to a function. To make these lines more readable, Python allows you to continue a statement onto the next line using a backslash (\). This is known as a line continuation character.

When Python encounters a backslash at the end of a line, it knows the current statement continues on the next line. This feature is especially useful for keeping code organized and preventing long lines that are difficult to read or maintain.

Example – Using Line Continuation

```
# Define several variables describing a product
name = "SuperVac 3000"
brand = "We Really Suck"
price = 199.99
warranty_years = 3
rating = 4.7

# basic use of line continuation
print("Product:", name,
      "\nBrand:", brand,
      "\nPrice: $", price,
      "\nWarranty:", warranty_years, "years",
      "\nRating:", rating, "stars")
```

This example uses a backslash at the end of each line to tell Python that the statement continues on the next line. It's useful when printing multiple variables or performing a long operation.

Example – Line Continuation with f-Strings

```
# Using line continuation with f-strings
name = "SuperVac 3000"
brand = "We Really Suck"
price = 199.99
rating = 4.7

print(f"Product: {name}\n" \
      f"Brand: {brand}\n" \
      f"Price: ${price:.2f}\n" \
      f"Rating: {rating} stars")
```

In this version, each line is an f-string, allowing for inline variable substitution while maintaining readability. The backslash joins all the lines into a single print statement. This technique is ideal for creating formatted, multi-line strings such as reports or user messages.

8. Building a Repeating System

Menu systems combine loops, conditionals, input, and functions. This makes programs more interactive and modular. The following program repeats until the user chooses to quit.

```
def menu():
    while True:
        print("\n1. Say Hello")
        print("2. Add Numbers")
        print("3. Quit")
        choice = input("Choose an option: ")

        if choice == "1":
            name = input("Enter your name: ")
            print(f"Hello, {name}!")
        elif choice == "2":
            a = int(input("Enter first number: "))
            b = int(input("Enter second number: "))
            print("Sum is", a + b)
        elif choice == "3":
            print("Goodbye!")
            break
        else:
            print("Invalid option!")
```

Practice Exercises

1) Echo Loop — Prompt the user repeatedly for text. Echo it back until they type 'stop' (case-insensitive). Trim whitespace before checking.

2) Basic Calculator — Build a menu (add, subtract, multiply, divide).

3) Tip Calculator — Ask for meal cost and tip percentage. Compute tip, total, and show both with two-decimal formatting.

4) Quiz Program — Ask at least 3 multiple-choice questions. Validate answers, track score, and print a result summary.

5) Name Tracker — Collect names until 'done'. Use a dictionary to count occurrences and then print each name with its frequency sorted by count.

6) System Expansion — Add at least two exercises as options to your menu system from this chapter.